

لغة ++C

C++ Program

فهرس المحتويات

1 مقدمة عن لغة ++C

2 تنصيب وتجهيز البيئة

3 أساسيات اللغة

4 أنواع البيانات

5 العوامل (Operators)

6 الجمل الشرطية وحلقات التكرار

7 المصفوفات (Arrays)

8 النصوص (Strings)

9 الدوال (Functions)

10 التحميل الزائد (Overloading)

11 المؤشرات (Pointers)

12 المراجع (References)

13 الهياكل (Structures)

14 البرمجة كائنية التوجه (OOP)

15 الوراثة (Inheritance)

16 تعدد الأشكال (Polymorphism)

17 القوالب (Templates)

18 مكتبة STL

1. مقدمة عن لغة ++C

++C هي لغة برمجة عالية المستوى متعددة الاستخدامات، طورها **Bjarne Stroustrup** عام 1985 كامتداد للغة C مع إضافة البرمجة كائنية التوجه (OOP). تتميز ++C بالأداء العالي والتحكم الكامل في إدارة الذاكرة.

مميزات ++C

- أداء عالٍ - لغة مصممة (Compiled) وسريعة جداً
- تحكم كامل بالذاكرة - إدارة يدوية للذاكرة عبر المؤشرات
- برمجة كائنية التوجه - تدعم OOP بشكل كامل
- متعددة الاستخدامات - ألعاب، أنظمة، تطبيقات، سيارات
- مكتبة STL ضخمة - مكتبة قياسية غنية بالخوارزميات وهياكل البيانات
- توافق مع C - يمكن تشغيل كود C داخل ++C
- قوالب (Templates) - برمجة عامة (Generic Programming)

مجالات استخدام ++C

- تطوير الألعاب (Unreal Engine, Unity)
- أنظمة التشغيل (Windows, Linux parts)
- برامج قواعد البيانات (MySQL, MongoDB)
- تطبيقات الوقت الحقيقي (Real-time Systems)
- الذكاء الاصطناعي وتعلم الآلة (TensorFlow C++ API)
- المتصفحات (Chrome, Firefox)
- الأنظمة المدمجة (Embedded Systems)
- تطبيقات السيارات والطيران

2. تنصيب وتجهيز البيئة

مترجمات ++C

الأمـر	المنصة	المترجم
g++ program.cpp -o program	Linux, macOS	GCC (g++)
clang++ program.cpp -o program	Linux, macOS	Clang (clang++)
cl program.cpp	Windows	MSVC

تثبيت على Linux

```
sudo apt update
sudo apt install g++ build-essential
g++ --version
```

تثبيت على Windows

حمل MinGW أو استخدم Visual Studio مع "Desktop development with C++".

تشغيل أول برنامج

```
#include <iostream>
using namespace std;

int main() {
    cout << "مرحباً بالعالم" << endl;
    return 0;
}
```

مرحباً بالعالم

3. أساسيات اللغة

هيكل برنامج ++C

```
#include <iostream> // مكتبة الإدخال/الإخراج
using namespace std; // استخدام مساحة الأسماء القياسية

int main() { // الدالة الرئيسية - نقطة بدء البرنامج
    cout << "Hello"; // طباعة نص
    return 0; // إنهاء البرنامج بنجاح
}
```

المتغيرات (Variables)

```
int x = 5; // عدد صحيح
double pi = 3.14159; // عدد عشري بدقة مضاعفة
float f = 3.14f; // عدد عشري
char c = 'A'; // حرف واحد
bool flag = true; // قيمة منطقية
string name = "Ahmed"; // نص (من STL)
```

قواعد تسمية المتغيرات

- يبدأ بحرف أو شرطة سفلية _
- يحتوي على أحرف، أرقام، وشرطات سفلية
- حساس لحالة الأحرف (age ≠ Age)
- لا يمكن استخدام الكلمات المحجوزة (int, return, if, etc).
- يفضل استخدام snake_case أو camelCase

الثوابت (Constants)

```
const double PI = 3.14159; // ثابت - لا يمكن تغيير قيمته
constexpr int MAX = 100; // ثابت وقت الترجمة (C++11)
```

الإدخال والإخراج

```
#include <iostream>
using namespace std;

int main() {
```

```
string name;  
int age;  
  
cout << "أدخل اسمك: ";  
cin >> name;  
  
cout << "أدخل عمرك: ";  
cin >> age;  
  
cout << "اسمك: " << name << ", عمرك: " << age << endl;  
return 0;  
}
```

التعليقات (Comments)

```
// هذا تعليق بسطر واحد  
  
/*  
هذا تعليق  
متعدد الأسطر  
*/
```

4. أنواع البيانات (Data Types)

النوع	الحجم	المدى	مثال
bool	1 بايت	true / false	<code>;bool b = true</code>
char	1 بايت	127 إلى 128-	<code>;char c = 'A'</code>
int	4 بايت	$2^{31}-1$ إلى 2^{31}	<code>;int x = 10</code>
unsigned int	4 بايت	0 إلى $2^{32}-1$	<code>;unsigned u = 42</code>
short	2 بايت	32767 إلى 32768-	<code>;short s = 100</code>
long	8 بايت	$2^{63}-1$ إلى 2^{63}	<code>;long l = 100000L</code>
float	4 بايت	~7 أرقام عشرية	<code>;float f = 3.14f</code>
double	8 بايت	~15 رقم عشري	<code>;double d = 3.14159</code>
long double	16 بايت	~19 رقم عشري	<code>;long double ld = 3.14L</code>

sizeof - معرفة حجم النوع

```
cout << "حجم int: " << sizeof(int) << " بايت" << endl;
cout << "حجم double: " << sizeof(double) << " بايت" << endl;
```

تحويل الأنواع (Type Casting)

```
// تحويل ضمني (Implicit)
double d = 10 / 3;           // 3.0 (قسمة صحيحة ثم تحويل)
double d2 = 10.0 / 3;        // 3.33333

// تحويل صريح (Explicit)
int x = (int)3.14;            // C-style: 3
int y = int(3.14);           // Function-style: 3
int z = static_cast<int>(3.14); // C++ style (موصى به)
```


5. العوامل (Operators)

العوامل الحسابية

```
int a = 10, b = 3;
a + b;    // 13 - جمع
a - b;    // 7  - طرح
a * b;    // 30 - ضرب
a / b;    // 3  - قسمة صحيحة (int / int)
a % b;    // 1  - باقي القسمة
```

العوامل المنطقية

```
bool a = true, b = false;
a && b;    // false - AND
a || b;    // true  - OR
!a;        // false - NOT
```

عوامل المقارنة

```
int x = 5, y = 10;
x == y;    // false - يساوي
x != y;    // true  - لا يساوي
x < y;     // true  - أصغر من
x > y;     // false - أكبر من
x <= y;    // true  - أصغر أو يساوي
x >= y;    // false - أكبر أو يساوي
```

عوامل الزيادة والنقصان

```
int x = 5;
x++;       // x = 6 (زيادة بعد الاستخدام)
++x;       // x = 7 (زيادة قبل الاستخدام)
x--;       // x = 6 (نقص بعد الاستخدام)
--x;       // x = 5 (نقص قبل الاستخدام)
```

عوامل الإسناد المركبة

```
x += 5;    // x = x + 5
x -= 3;    // x = x - 3
x *= 2;    // x = x * 2
```


6. الجمل الشرطية وحلقات التكرار

if / else if / else

```
int score = 85;

if (score >= 90) {
    cout << "ممتاز";
} else if (score >= 70) {
    cout << "جيد جداً";
} else if (score >= 50) {
    cout << "مقبول";
} else {
    cout << "راسب";
}
```

العامل الثلاثي (Ternary Operator)

```
int x = 10;
string result = (x > 0) ? "موجب" : "سالب";
cout << result; // موجب
```

switch

```
char grade = 'B';
switch (grade) {
    case 'A':
        cout << "ممتاز";
        break;
    case 'B':
        cout << "جيد جداً";
        break;
    case 'C':
        cout << "جيد";
        break;
    default:
        cout << "غير معروف";
}
```

حلقة for

```
for (int i = 0; i < 5; i++) {
    cout << i << " "; // 0 1 2 3 4
}

// for مصفوفة
int arr[] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
```

```
        cout << arr[i] << " ";
    }

    // Range-based for (C++11)
    for (int num : arr) {
        cout << num << " ";
    }
}
```

حلقة while

```
int i = 0;
while (i < 5) {
    cout << i << " ";
    i++;
}

// do-while (تنفذ مرة على الأقل)
int j = 0;
do {
    cout << j << " ";
    j++;
} while (j < 5);
```

continue و break

```
for (int i = 0; i < 10; i++) {
    if (i == 5) break;           // يخرج من الحلقة عند 5
    if (i == 2) continue;       // يتخطى i==2
    cout << i << " ";           // 0 1 3 4
}
```

7. المصفوفات (Arrays)

مصفوفة أحادية البعد

```
// تعريف وتهئية
int arr[5]; // مصفوفة من 5 أعداد (غير مهيأة)
int arr2[5] = {1, 2, 3, 4, 5}; // تهئية كاملة
int arr3[] = {1, 2, 3, 4, 5}; // الحجم يُستنتج تلقائياً
int arr4[5] = {1, 2}; // الباقي = 0 → {1,2,0,0,0}

// الوصول للعناصر
cout << arr2[0]; // 1
arr2[2] = 10; // تعديل

// التكرار على المصفوفة
for (int i = 0; i < 5; i++) {
    cout << arr2[i] << " ";
}
```

مصفوفة ثنائية البعد

```
// مصفوفة 4x3
int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

cout << matrix[1][2]; // 7

// التكرار على مصفوفة ثنائية
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        cout << matrix[i][j] << "\t";
    }
    cout << endl;
}
```

حجم المصفوفة

```
int arr[] = {10, 20, 30, 40, 50};
int size = sizeof(arr) / sizeof(arr[0]); // 5
cout << "عدد العناصر: " << size;
```

8. النصوص (Strings)

في ++C يوجد نوعان من النصوص: C-strings (مصفوفة محارف) و std::string (مكتبة STL).

```
#include <iostream>
#include <string>
using namespace std;
```

C++ String (std::string)

```
string s1 = "Hello";
string s2("World");
string s3 = s1 + " " + s2;    // دمج النصوص - "Hello World"

cout << s3.length() << endl;    // الطول - 11
cout << s3.size() << endl;    // الطول - 11
cout << s3[0] << endl;    // أول حرف - 'H'
cout << s3.at(1) << endl;    // 'e' (مع فحص الحدود)

// دوال مهمة
s3.substr(0, 5);    // استخراج نص فرعي - "Hello"
s3.find("World");    // إيجاد موقع نص - 6
s3.replace(0, 5, "Hi");    // استبدال - "Hi World"
s3.insert(5, " C++");    // إدراج نص
s3.erase(0, 3);    // حذف أحرف
s3.empty();    // هل النص فارغ؟
s3.clear();    // تفريغ النص

// تحويل لحروف كبيرة/صغيرة
for (char &c : s3) c = toupper(c);
```

C-String (موروث من C)

```
char name[] = "Ahmed";    // مصفوفة محارف تنتهي بـ 0\
char name2[20] = "Sara";

cout << strlen(name);    // الطول - 5
strcpy(name2, name);    // نسخ
strcat(name2, "!");    // دمج
strcmp(name, "Ahmed");    // متساويان -> 0
```

9. الدوال (Functions)

تعريف واستدعاء دالة

```
// تعريف الدالة
int add(int a, int b) {
    return a + b;
}

// استدعاء الدالة
int result = add(5, 3); // 8
```

النموذج (Function Prototype)

```
// إعلان مسبق (Prototype) - قبل main()
int multiply(int a, int b);

int main() {
    cout << multiply(4, 5); // 20
    return 0;
}

// تعريف الدالة بعد main()
int multiply(int a, int b) {
    return a * b;
}
```

معاملات افتراضية (Default Arguments)

```
void greet(string name, string greeting = "مرحباً") {
    cout << greeting << " " << name << endl;
}

greet("أحمد"); // مرحباً أحمد
greet("أحمد", "أهلاً"); // أهلاً أحمد
```

التمرير بالقيمة والمرجع

```
// تمرير بالقيمة (نسخة) - لا يؤثر على الأصل
void passByValue(int x) {
    x = 100;
}

// تمرير بمرجع - يؤثر على الأصل
void passByReference(int &x) {
    x = 100;
}

// تمرير بمرجع ثابت (للقراءة فقط) - لتحسين الأداء
```

```
void printString(const string &s) {  
    cout << s;  
}  
  
int a = 10;  
passByValue(a);    // a يزال 10  
passByReference(a); // a أصبح 100
```

التحميل الزائد للدوال (Function Overloading)

```
// دوال بنفس الاسم لكن معاملات مختلفة  
int sum(int a, int b) {  
    return a + b;  
}  
  
double sum(double a, double b) {  
    return a + b;  
}  
  
int sum(int a, int b, int c) {  
    return a + b + c;  
}  
  
cout << sum(3, 4);           // 7 (int version)  
cout << sum(3.5, 2.5);       // 6.0 (double version)  
cout << sum(1, 2, 3);        // 6 (three params)
```

الدوال المضمنة (Inline Functions)

```
inline int square(int x) {  
    return x * x;  
}  
// inline على المترجم نسخ الكود مكان الاستدعاء لتسريع التنفيذ
```

التكرار (Recursion)

```
// دالة عاملي (Factorial)  
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}  
  
cout << factorial(5); // 120
```


10. التحميل الزائد (Overloading)

التحميل الزائد للعوامل (Operator Overloading)

```
class Complex {
public:
    double real, imag;

    Complex(double r = 0, double i = 0) : real(r), imag(i) {}

    // تحميل عامل +
    Complex operator+(const Complex &c) {
        return Complex(real + c.real, imag + c.imag);
    }

    // تحميل عامل >> (للطباعة)
    friend ostream &operator<<(ostream &out, const Complex &c) {
        out << c.real << " + " << c.imag << "i";
        return out;
    }
};

Complex c1(3, 4), c2(1, 2);
Complex c3 = c1 + c2;    // 4 + 6i
cout << c3;
```

11. المؤشرات (Pointers)

المؤشر (Pointer) هو متغير يخزن عنوان ذاكرة. من أقوى وأخطر ميزات C++.

```
int x = 10;
int *ptr = &x;           // ptr يخزن عنوان x

cout << x;                // القيمة - 10
cout << &x;              // العنوان - 0x7fff...
cout << ptr;              // نفس العنوان - 0x7fff...
cout << *ptr;             // ptr قيمة ما يشير إليه - 10 (Dereference)

*ptr = 20;                // عبر المؤشر x تغير قيمة
cout << x;                // 20
```

المؤشر والقوائم (Pointer Arithmetic)

```
int arr[] = {10, 20, 30, 40, 50};
int *ptr = arr;           // ptr لأول عنصر

cout << *ptr;             // 10
ptr++;                    // ينتقل للعنصر التالي (زيادة 4 بايت)
cout << *ptr;             // 20
ptr += 2;                 // ينتقل للعنصر الثالث من الموقع الحالي
cout << *ptr;             // 40

// التكرار على مصفوفة باستخدام المؤشرات
for (int *p = arr; p < arr + 5; p++) {
    cout << *p << " ";
}
```

المؤشر الفارغ (Null Pointer)

```
int *ptr = nullptr;       // مؤشر لا يشير لشيء - C++11
// int *ptr = 0; أو int *ptr = NULL; قبل C++11

if (ptr == nullptr) {
    cout << "مؤشر فارغ";
}
```

الذاكرة الديناميكية (new / delete)

```
// تخصيص ذاكرة ديناميكية
int *p = new int;         // واحد int تخصيص
*p = 42;
cout << *p;               // 42
delete p;                 // تحرير الذاكرة - مهم جداً!

// تخصيص مصفوفة ديناميكية
int *arr = new int[10];   // مصفوفة من 10 أعداد
```


12. المراجع (References)

المراجع (Reference) هو اسم مستعار (Alias) لمتغير موجود. يشبه المؤشر لكنه أكثر أماناً.

```
int x = 10;
int &ref = x;           // ref هو مرجع لـ x

ref = 20;                // x 20 يصبح
cout << x;              // 20
cout << ref;            // 20

// الفرق بين المرجع والمؤشر
// 1. المرجع يجب أن يُهيأ عند تعريفه.
// 2. المرجع لا يمكن تغييره ليشير لشيء آخر.
// 3. nullptr المرجع لا يمكن أن يكون.
// 4. المرجع يستخدم مثل المتغير العادي (بدون * أو &).
```

13. الهياكل (Structures)

```
struct Student {
    string name;
    int age;
    double gpa;
};

// إنشاء واستخدام
Student s1;
s1.name = "أحمد";
s1.age = 20;
s1.gpa = 3.8;

Student s2 = {"سارة", 22, 3.9}; // تهيئة مباشرة
cout << s1.name << " - GPA: " << s1.gpa;
```

الفرق بين struct و class

- في struct: الأعضاء عامة (public) افتراضياً
- في class: الأعضاء خاصة (private) افتراضياً
- باقي المميزات متطابقة

14. البرمجة كائنية التوجه (OOP)

الفئة والكائن (Class & Object)

```
class Car {
private:                                // خاص - لا يمكن الوصول من الخارج
    string brand;
    int year;
    double speed;

public:                                  // عام - يمكن الوصول من الخارج
    // Constructor (دالة البناء)
    Car(string b, int y) : brand(b), year(y), speed(0) {}

    // Setters / Getters (Encapsulation - التغليف)
    void setSpeed(double s) {
        if (s >= 0) speed = s;
    }

    double getSpeed() {
        return speed;
    }

    void displayInfo() {
        cout << brand << " (" << year << ") - سرعة: " << speed << " كم/س";
    }
};

// إنشاء كائن |
Car myCar("Toyota", 2022);
myCar.setSpeed(120);
myCar.displayInfo();
// myCar.speed = 200; // خطأ! speed خاص (private)
```

دالة البناء والمدمر (Constructor & Destructor)

```
class Player {
private:
    string name;
    int health;

public:
    // Constructor افتراضي
    Player() : name("لاعب"), health(100) {
        cout << "تم إنشاء لاعب" << endl;
    }

    // Constructor بمعاملات
    Player(string n, int h) : name(n), health(h) {}

    // Copy Constructor
    Player(const Player &p) : name(p.name), health(p.health) {}

    // Destructor (مدمر) - ينفذ عند حذف الكائن
    ~Player() {
        cout << name << " : تم تدمير اللاعب" << endl;
    }
};
```

```
    }  
};
```

قائمة التهيئة (Initializer List)

```
class Point {  
private:  
    int x, y;  
    const int id;          // الثوابت يجب تهيئتها في قائمة التهيئة  
  
public:  
    Point(int a, int b, int i) : x(a), y(b), id(i) {}  
    // داخل الدالة: x = a; y = b;  
};
```

this

```
class Person {  
    string name;  
public:  
    Person(string name) {  
        this->name = name; // variable و parameter تميز بين  
    }  
  
    Person &setName(string name) {  
        this->name = name;  
        return *this;      // Fluent interface (chaining)  
    }  
};
```

15. الوراثة (Inheritance)

وراثة بسيطة

```
class Animal {
protected:                                // protected: مرئي للفئات المشتقة
    string name;
    int age;

public:
    Animal(string n, int a) : name(n), age(a) {}

    virtual void speak() {
        cout << "الحيوان يصدر صوتاً" << endl;
    }
};

// Dog يرث من Animal
class Dog : public Animal {
public:
    Dog(string n, int a) : Animal(n, a) {} // الأب constructor استدعاء

    void speak() override {                // تطليل (Override)
        cout << name << " يقول : Woof!" << endl;
    }

    void fetch() {                          // دالة جديدة
        cout << name << " يجلب الكرة" << endl;
    }
};

Dog dog("Buddy", 3);
dog.speak(); // Buddy يقول : Woof!
dog.fetch(); // Buddy يجلب الكرة!
```

أنواع الوراثة

نوع الوراثة	public في الأب	protected في الأب	private في الأب
public	public	protected	مخفي
protected	protected	protected	مخفي
private	private	private	مخفي

16. تعدد الأشكال (Polymorphism)

الدوال الافتراضية (Virtual Functions)

```
class Shape {
public:
    virtual double area() = 0;    // دالة افتراضية خالصة (Pure Virtual)
    virtual void draw() {        // دالة افتراضية (يمكن تطليلها)
        cout << "رسم شكل" << endl;
    }
    virtual ~Shape() {}          // Destructor (هام!)
};
```

```
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}

    double area() override {
        return 3.14159 * radius * radius;
    }

    void draw() override {
        cout << "رسم دائرة" << endl;
    }
};
```

```
class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}

    double area() override {
        return width * height;
    }
};
```

```
// تعدد الأشكال - نفس الدالة تتصرف بشكل مختلف
Shape *shapes[] = {new Circle(5), new Rectangle(4, 6)};
for (Shape *s : shapes) {
    s->draw();
    cout << "المساحة: " << s->area() << endl;
    delete s; // تسريب ذاكرة -> virtual غير destructor لو
}
```

17. القوالب (Templates)

القوالب تسمح بكتابة كود عام (Generic) يعمل مع أي نوع بيانات.

قالب دالة (Function Template)

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

cout << max(3, 7);           // 7 (int)
cout << max(3.5, 2.1);       // 3.5 (double)
cout << max("Ahmed", "Sara"); // Sara (string)
```

قالب فئة (Class Template)

```
template <typename T>
class Box {
private:
    T value;
public:
    Box(T v) : value(v) {}
    T getValue() { return value; }
    void setValue(T v) { value = v; }
};

Box<int> intBox(42);
Box<string> strBox("Hello");
cout << intBox.getValue(); // 42
cout << strBox.getValue(); // Hello
```

معاملات قالب متعددة

```
template <typename T, typename U>
class Pair {
private:
    T first;
    U second;
public:
    Pair(T f, U s) : first(f), second(s) {}
    T getFirst() { return first; }
    U getSecond() { return second; }
};

Pair<string, int> p("أحمد", 25);
cout << p.getFirst() << ": " << p.getSecond();
```

18. مكتبة STL (Standard Template Library)

مكتبة STL هي مجموعة من هياكل البيانات والخوارزميات الجاهزة. تتكون من:

- الحاويات (Containers) - vector, list, map, set, etc.
- الخوارزميات (Algorithms) - sort, find, search, etc.
- المكررات (Iterators) - للتنقل بين عناصر الحاويات

الحاويات الأساسية

الحاوية	الوصف	مكتبة
vector	مصفوفة ديناميكية	<vector>
list	قائمة مترابطة ثنائياً	<list>
deque	طابور ثنائي النهاية	<deque>
stack	كومة (LIFO)	<stack>
queue	طابور (FIFO)	<queue>
priority_queue	طابور أولوي	<queue>
set	مجموعة (عناصر فريدة مرتبة)	<set>
map	قاموس (مفتاح - قيمة)	<map>
unordered_set	مجموعة غير مرتبة	<unordered_set>
unordered_map	قاموس غير مرتب	<unordered_map>

19. المتجهات (Vectors)

```
#include <vector>
using namespace std;

vector<int> vec; // متجه فارغ
vector<int> vec2(5); // 5 عناصر بقيمة 0
vector<int> vec3(5, 10); // 5 عناصر بقيمة 10
vector<int> vec4 = {1, 2, 3, 4, 5}; //تهيئة مباشرة (C++11)

// إضافة وحذف
vec.push_back(10); // إضافة في النهاية
vec.pop_back(); // حذف آخر عنصر
vec.insert(vec.begin() + 2, 99); // إدراج في موقع
vec.erase(vec.begin() + 1); // حذف من موقع

// الوصول للعناصر
cout << vec[0]; // بدون فحص حدود
cout << vec.at(0); // مع فحص حدود (يرمي استثناء)
cout << vec.front(); // أول عنصر
cout << vec.back(); // آخر عنصر

// معلومات
vec.size(); // عدد العناصر الحالي
vec.capacity(); // السعة المخصصة
vec.empty(); // هل فارغ؟

// التكرار
for (int i = 0; i < vec.size(); i++) {
    cout << vec[i];
}

// Range-based for
for (int x : vec) {
    cout << x;
}

// باستخدام Iterator
for (vector<int>::iterator it = vec.begin(); it != vec.end(); ++it) {
    cout << *it;
}
```

map (القاموس)

```
#include <map>

map<string, int> ages;
ages["أحمد"] = 25;
ages["سارة"] = 30;
ages.insert({"خالد", 22});

cout << ages["أحمد"]; // 25

for (const auto &pair : ages) {
    cout << pair.first << ": " << pair.second << endl;
}
```

```
#include <set>

set<int> s;
s.insert(3);
s.insert(1);
s.insert(4);
s.insert(1);           // لن يضاف (مكرر)

cout << s.count(1);     // موجود 1

for (int x : s) {
    cout << x << " ";   // 1 3 4 (مرتبة)
}
```

الخوارزميات (Algorithms)

```
#include <algorithm>

vector<int> v = {5, 2, 8, 1, 9};

sort(v.begin(), v.end());           // ترتيب تصاعدي
sort(v.rbegin(), v.rend());         // ترتيب تنازلي
reverse(v.begin(), v.end());        // عكس الترتيب

auto it = find(v.begin(), v.end(), 8); // إيجاد عنصر
if (it != v.end()) cout << *it;

int minVal = *min_element(v.begin(), v.end());
int maxVal = *max_element(v.begin(), v.end());

count(v.begin(), v.end(), 5);       // عدد تكرارات 5
fill(v.begin(), v.end(), 0);        // ملء الكل بـ 0

// remove_if مع lambda (C++11)
remove_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
```

20. معالجة الأخطاء (Exceptions)

```
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) {
        throw runtime_error("خطأ: القسمة على صفر");
    }
    return a / b;
}

try {
    double result = divide(10, 0);
    cout << result;
} catch (const runtime_error &e) {
    cout << "استثناء: " << e.what();
} catch (...) {
    cout << "حدث خطأ غير معروف";
}
```

أنواع الاستثناءات الشائعة

السبب	المكتبة	الاستثناء
خطأ وقت التشغيل	<stdexcept>	runtime_error
خطأ منطقي	<stdexcept>	logic_error
مؤشر خارج الحدود	<stdexcept>	out_of_range
معامل غير صالح	<stdexcept>	invalid_argument
فشل تخصيص ذاكرة	<new>	bad_alloc

21. التعامل مع الملفات

```
#include <fstream>

// كتابة ملف
ofstream outFile("example.txt");
if (outFile.is_open()) {
    outFile << "Hello, World!" << endl;
    outFile << "السطر الثاني" << endl;
    outFile.close();
}

// قراءة ملف
ifstream inFile("example.txt");
string line;
while (getline(inFile, line)) {
    cout << line << endl;
}
inFile.close();

// أنماط فتح الملفات
// ios::in - للقراءة
// ios::out - للكتابة (مسح المحتوى)
// ios::app - إضافة للنهاية
// ios::binary - وضع ثنائي

fstream file("data.txt", ios::in | ios::out | ios::app);
```

22. دوال لامدا (Lambda Functions) - C++11

```
// صيغة لامدا: [capture](parameters) -> return_type { body }
```

```
// لامدا بسيطة
```

```
auto square = [](int x) { return x * x; };  
cout << square(5); // 25
```

```
// مع نوع الإرجاع
```

```
auto divide = [](double a, double b) -> double {  
    if (b == 0) return 0;  
    return a / b;  
};
```

```
// Capture by value
```

```
int x = 10;  
auto printX = [x]() { cout << x; }; // ينسخ x
```

```
// Capture by reference
```

```
auto modifyX = [&x]() { x = 20; };
```

```
// كل شيء
```

```
[=]() { ... } // كل المتغيرات بالقيمة  
[&]() { ... } // كل المتغيرات بالمرجع
```

```
// استخدام مع الخوارزميات
```

```
vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8};  
int count = count_if(v.begin(), v.end(), [](int n) {  
    return n % 2 == 0;  
});  
cout << count; // 4
```

```
// ترتيب مخصص
```

```
sort(v.begin(), v.end(), [](int a, int b) {  
    return a > b; // تنازلي  
});
```


23. المؤشرات الذكية (Smart Pointers) - C++11

المؤشرات الذكية تدير الذاكرة تلقائياً، مما يمنع التسريبات (Memory Leaks).

unique_ptr

```
#include <memory>

// unique_ptr - ملكية حصرية (لا يمكن نسخه)
unique_ptr<int> p1 = make_unique<int>(42);
cout << *p1; // 42

// unique_ptr<int> p2 = p1; // خطأ! لا يمكن النسخ
unique_ptr<int> p2 = move(p1); // فقط يمكن النقل (move)
// بعد النقل p1 = nullptr
```

shared_ptr

```
// shared_ptr - ملكية مشتركة (عدّاد مراجع)
shared_ptr<int> s1 = make_shared<int>(100);
shared_ptr<int> s2 = s1; // الآن count = 2

cout << s1.use_count(); // 2
s2.reset(); // count = 1
```

مثال عملي مع الفئات

```
class Resource {
public:
    Resource() { cout << "تم تخصيص مورد" << endl; }
    ~Resource() { cout << "تم تحرير المورد" << endl; }
    void work() { cout << "...العمل على المورد" << endl; }
};

{
    unique_ptr<Resource> res = make_unique<Resource>();
    res->work();
    // عند الخروج من النطاق، يُحذف تلقائياً
} // "تم تحرير المورد"
```

24. أفضل الممارسات

نصائح لكتابة كود ++C جيد

- استخدم `const` حيثما أمكن
- استخدم `nullptr` بدلاً من `NULL` أو `0`
- فضل `std::array` أو `std::vector` على المصفوفات التقليدية
- استخدم `auto` لتقليل التكرار (ولكن ليس على حساب الوضوح)
- استخدم المؤشرات الذكية (`smart pointers`) بدلاً من `raw pointers`
- اكتب `Destructor` افتراضياً (`virtual`) للفئات الأساسية
- استخدم `override` عند تظليل دوال افتراضية
- استخدم `constexpr` للثوابت وقت الترجمة
- قم بتحرير الذاكرة التي تخصصها (`new/delete`, `new/delete[]`)
- استخدم `namespace` لتجنب تضارب الأسماء

قواعد تسمية متفق عليها

- الفئات: `class MyClass` → `PascalCase`
- الدوال: `void myFunction()` → `camelCase`
- المتغيرات: `int my_variable` → `snake_case`
- الثوابت: `const int MAX_SIZE` → `UPPER_CASE`
- مؤشرات: `int *pValue` → `prefixed with p`

معايير ++C الحديثة

المعيار	السنة	أهم الميزات
C++98	1998	المعيار الأول
C++11	2011	<code>auto</code> , <code>nullptr</code> , <code>lambda</code> , <code>smart pointers</code> , <code>move semantics</code>
C++14	2014	<code>generic lambdas</code> , <code>return type deduction</code>
C++17	2017	<code>if constexpr</code> , <code>structured bindings</code> , <code>filesystem</code>

concepts, ranges, coroutines, modules	2020	C++20
std::print, std::expected, deducing this	2023	C++23

مشاريع مقترحة للمبتدئين

- آلة حاسبة (Console Calculator)
- مدير جهات اتصال (Contact Manager)
- لعبة X0 (Tic-Tac-Toe)
- نظام إدارة مكتبة صغير
- مشغل موسيقى بسيط
- محلل نصوص (Word/Character Counter)
- لعبة Snake (الثعبان)
- تطبيق بحث وترتيب بيانات

مصادر تعليمية

- [cppreference.com](#) - المرجع الرسمي
- [learncpp.com](#) - دورة تعليمية مجانية شاملة
- [isocpp.org](#) - موقع اللجنة المعيارية
- [cplusplus.com](#) - مرجع وأمثلة
- [LeetCode / HackerRank](#) - حل مشاكل برمجية
- [GitHub](#) - استكشاف مشاريع مفتوحة المصدر